

Open-Loop Gait Optimization and On-Device Sinusoidal Firmware for the 12-DOF Hexapod

E90 Hexapod Team

April 13, 2026

Abstract

This note documents the *no-IMU* gait optimization pipeline implemented in Python and its deployment on the Pololu Maestro-driven hexapod via the `hexapod_openloop_sine` Arduino sketch. The optimizer searches over 25 continuous parameters describing per-leg sinusoidal abduction and flexion with tripod phase coupling, using a reduced-order body model integrated with `scipy.integrate.solve_ivp`. The firmware reproduces the same command law as the script `leg_commands()`, maps optimizer angles to hardware joint commands, and applies safety clamps before commanding servos. IMU feedback and online adaptation are explicitly out of scope here.

1 Purpose and scope

The goal is to align three artifacts:

1. a **simulation-based optimizer** that improves open-loop gait parameters without inertial sensing;
2. a **JSON summary** of the best parameter vector and diagnostics; and
3. **embedded firmware** that runs the identical sinusoidal gait on the physical robot.

The Python reference script is `hexapod_no_imu_optimization.py`. It lives in `ode_optimization_no_imu/`, next to the roadmap notes. The Teensy sketch and parameter header are in subdirectory `hexapod_openloop_sine/` under `firmware/`.

2 Kinematic command law (optimizer and firmware)

2.1 Leg indexing and tripod coupling

Legs are ordered consistently in both Python and firmware (Maestro channel order):

$$\text{FL, FR, ML, MR, RL, RR.}$$

Tripod A is {FL, MR, RL}; tripod B is {FR, ML, RR}. The base phase offset for tripod B is π radians relative to tripod A, matching `TRIPOD_PHASE` in Python and `TRIPOD_OFFSET_RAD` in firmware.

2.2 Sinusoidal joint commands

Let $\varphi(t) = 2\pi ft$ with gait frequency f in Hz. For leg index $i \in \{0, \dots, 5\}$, define the leg carrier phase

$$\psi_i(t) = \varphi(t) + \delta_i + s_i^{\text{abd}}, \quad \psi'_i(t) = \varphi(t) + \delta_i + s_i^{\text{flx}},$$

where $\delta_i \in [0, \pi]$ is the tripod offset and $s_i^{\text{abd}}, s_i^{\text{flx}}$ are abduction and flexion phase shifts (stored in degrees in the parameter file, converted to radians in software).

The optimizer works in radians internally; amplitudes for abduction and flexion are fixed per leg from default tables `DEFAULT_ABD_AMP_DEG` and `DEFAULT_FLX_AMP_DEG` (converted to radians in Python). The commanded angles are

$$\theta_i^{\text{abd}}(t) = \theta_i^{\text{abd,rest}} + A_i^{\text{abd}} \sin(\psi_i(t)), \tag{1}$$

$$\theta_i^{\text{flx}}(t) = \theta_i^{\text{flx,rest}} + A_i^{\text{flx}} \sin(\psi'_i(t)). \tag{2}$$

The firmware evaluates the equivalent expressions using degrees for rests, shifts, and amplitudes, then maps to abstract integer joint commands (Section 4).

2.3 Optimization vector

The decision vector has 25 scalars:

- $\theta^{\text{abd,rest}} \in \mathbb{R}^6$ (degrees),
- $\theta^{\text{flx,rest}} \in \mathbb{R}^6$ (degrees),
- $s^{\text{abd}} \in \mathbb{R}^6$ (degrees),
- $s^{\text{flx}} \in \mathbb{R}^6$ (degrees),

- $f \in \mathbb{R}$ (Hz).

Amplitudes A_i^{abd} , A_i^{flx} are *not* part of this vector; they match the Python defaults unless you change the source.

3 Reduced-order simulation model (Python)

The script integrates an eight-state planar body abstraction (longitudinal position and velocity, vertical position and velocity, roll and pitch and their rates). Leg forces depend on soft contact weighting from flexion commands, static support, damping, ground stiffness, and a stride thrust term proportional to abduction rate. The objective is a weighted sum:

- **Primary:** maximize mean forward speed in the steady-state window (implemented as a negative penalty on mean $\dot{x}$).
- **Penalties:** backward motion, roll and pitch RMS, vertical oscillation, a servo usage proxy, contact imbalance, and quadratic regularization toward the default parameter vector.

Optimization uses `scipy.optimize.minimize` with method L-BFGS-B, box constraints, multiple random restarts, and configurable weights (`OptimizationConfig`). Outputs include summary JSON, time series CSV, and optional plots.

This model is an engineering surrogate for screening gait parameters; it is not a full multibody simulation. Agreement with hardware requires calibration of the degree-to-pulse mapping (Section 4).

4 Firmware mapping to hardware

4.1 Abstract joint commands and Maestro

As in `hexapod_walking_v2.ino`, each servo receives a pulse width in microseconds:

$$u_k = \text{clamp}(c_k + d_k q_k),$$

where c_k is `servoCenter[k]`, $d_k \in \{-1, +1\}$ is `servoDir[k]`, and q_k is the integer joint command. Values are clamped to `MIN_US` and `MAX_US` before being sent with the Pololu compact protocol on `Serial1`.

4.2 Degree-to-command scaling

The sketch introduces two tunable scales, k_{abd} and k_{flx} (`ABD_CMD_PER_DEG`, `FLX_CMD_PER_DEG`), such that

$$q_i^{\text{abd}} = \text{round}(k_{\text{abd}} \tilde{\theta}_i^{\text{abd}}), \quad q_i^{\text{flx}} = \text{round}(k_{\text{flx}} \tilde{\theta}_i^{\text{flx}}),$$

where $\tilde{\theta}$ denotes the degree-valued sine law (rest plus amplitude times sine). Before `jointToUs`, commands pass through asymmetric bounds (`ABD_CMD_MIN/MAX`, `FLX_CMD_MIN/MAX`) to limit excursion on hardware.

The scales are *not* produced by the ODE optimizer; they must be tuned once so that optimized angles map to safe, effective motion on the physical robot. The defaults in the sketch ($k_{\text{abd}} = 12$, $k_{\text{flx}} = 42$) are starting points informed by the order of magnitude of the legacy v2 tripod joint commands.

4.3 Timing and startup

The main loop updates the phase integrator on a non-blocking schedule (`INTERP_DELAY_MS`, default 10 ms), with Δt capped to avoid large jumps after pauses. Gait frequency can be linearly ramped from zero to f over `FREQ_RAMP_MS` (default 3000 ms) to reduce the severity of motion at power-on.

4.4 Parameter injection from optimization results

File `optimized_gait_params.h` holds `OPT_*` arrays and `OPT_FREQUENCY_HZ`. After a Python run, copy `optimized_parameters` from `*_summary.json` into this header (rests, shifts, frequency) and rebuild. Amplitudes in the header must match the optimizer’s fixed tables unless you change both Python and firmware consistently.

5 Verification workflow

1. Run the optimizer script (or equivalent) and archive the emitted summary JSON.
2. Update `optimized_gait_params.h` and, if needed, k_{abd} , k_{flx} , and command clamps.
3. On the bench, confirm tripod alternation, absence of sustained servo saturation, and stable behavior before unattended operation.
4. Optionally compare logged joint commands against Python `leg_commands()` on a common time grid for the same parameters.

6 Limitations and next steps

- **No IMU:** this firmware does not close the loop on attitude; disturbances are not compensated here.
- **Model mismatch:** optimized parameters are only as good as the reduced ODE and the hardware mapping.
- **Future work:** IMU reflex layer on top of this open loop; EEPROM or host push of optimized vectors; optional conversion script from JSON to `.h` to avoid manual transcription errors.

Repository paths

- **Optimizer:** `ode_optimization_no_imu/hexapod_no_imu_optimization.py`
- **Firmware:** `hexapod_openloop_sine.ino`, `optimized_gait_params.h`
- **Discrete baseline:** `hexapod_walking_v2/hexapod_walking_v2.ino`